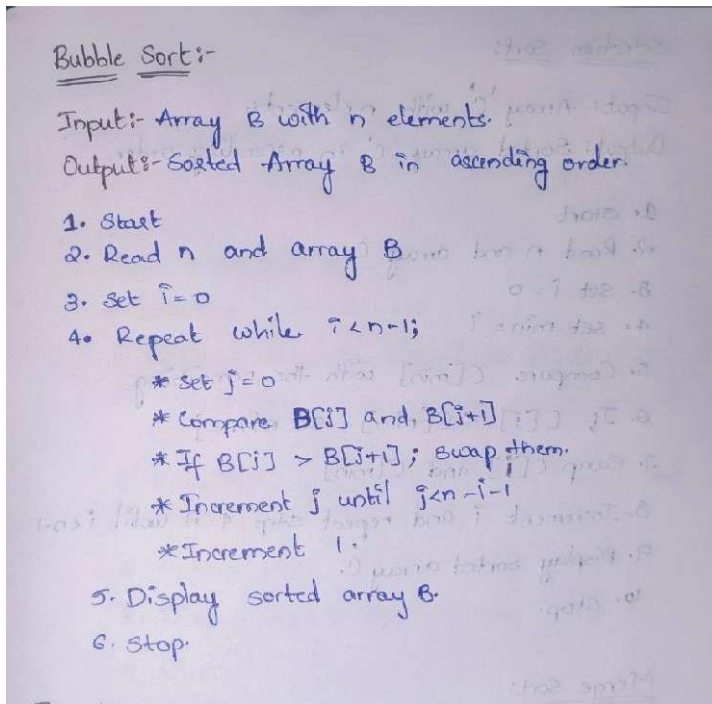


PRACTICAL: 01

Aim: To implement and find the time analysis of sorting algorithms. Bubble sort, Selection sort, Insertion sort, Merge sort and Quick sort.

A) Bubble Sort:

Algorithm:



Program / Code:

```
#include<iostream>

using namespace std;

int main()
{
    int a[60], size, i, j, temp;

    cout<<"Enter the size of the array: ";
    cin>>size;

    cout<<"Enter the array elements: ";

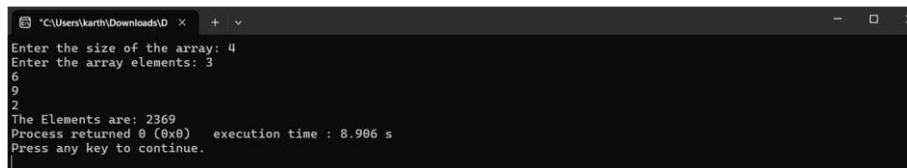
    for(i=0; i<size; i++){
```

```
        cin>>a[i];
    }
    for(i=0; i<size-1; i++){
        for(j=0; j<size-i-1; j++){
            if(a[j]>a[j+1]){
                temp = a[j];
                a[j] = a[j+1];
                a[j+1] = temp;
            }
        }
    }

    cout<<"The Elements are: ";
    for(i=0; i<size; i++){
        cout<<a[i];
    }

    return 0;
}
```

Output:



```
"C:\Users\karth\Downloads\D" x + v - □ x
Enter the size of the array: 4
Enter the array elements: 3
6
9
2
The Elements are: 2369
Process returned 0 (0x0)   execution time : 8.906 s
Press any key to continue.
```

Time Complexity:

Bubble Sort:

```

for (i=0; i<n-1; i++) {
    for (j=0; j<n-i-1; j++) {
        if (a[j] > a[j+1]) {
            temp = a[j];
            a[j] = a[j+1];
            a[j+1] = temp;
        }
    }
}

```

$T(n) = (n-1) + (n-2) + \dots + 1$
 $T(n) = \frac{n(n-1)}{2}$
 $T(n) = \frac{n^2 - n}{2}$
 $T(n) = O(n^2)$

→ Best case = Average case = worst case = $O(n^2)$

Best case: The array is sorted + Average case:

No. of Comparisons: $(n-1) + (n-2) + (n-3) + \dots + 2 + 1$

$T(n) = n + (n-1) + (n-2) + \dots + 1$
 $T(n) = O(n^2)$

Worst case: The array is in reverse order

$(n-1) + (n-2) + (n-3) + \dots + 2 + 1$
 $1 + 2 + 3 + \dots + (n-1)$
 $= n(n-1)/2$
 $T(n) = n(n-1)/2$
 $T(n) = O(n^2)$

B) Insertion Sort:

Insertion Sort:

Input: Array A with n elements
Output: Sorted array A in ascending order

1. Start
2. Read n and array A.
3. Set i = 1
4. Set Key = A[i] and j = i-1
5. While j >= 0 and A[j] > Key, shift A[j] to A[j+1]
6. Insert Key at A[j+1]
7. Increment i and repeat steps 4-6 until i = n
8. Display sorted array A.
9. Stop.

Program / Code:

```
#include<iostream>

using namespace std;

int main(){

    int a[60], size, i, j, min, temp;

    cout<<"Enter the size of the array: ";

    cin>>size;

    cout<<"Enter the array elements: ";

    for(i=0; i<size; i++){

        cin>>a[i];

    }

    for(i=0; i<size-1; i++){

        min = i;

        for(j=i+1; j<size; j++){

            if(a[j] < a[min]){

                min = j;

            }

        }

        if(min != i){

            temp = a[i];

            a[i] = a[min];

            a[min] = temp;

        }

    }

    cout<<"The Elements are: ";

    for(i=0; i<size; i++){

        cout<<a[i]<<" ";

    }

}
```

```

    }

    return 0;

}

```

Output:

```

C:\Users\karth\Downloads\D x + v
Enter the size of the array: 4
Enter the array elements: 6
3
9
2
The Elements are: 2 3 6 9
Process returned 0 (0x0)   execution time : 19.363 s
Press any key to continue.

```

Time Complexity:

Insertion Sort

```

for (i=1; i<n; i++) {
    Key = arr[i]
    j = i - 1
    while (j > 0 && arr[j] > Key) {
        arr[j+1] = arr[j]
        j = j - 1
    }
    arr[j+1] = Key
}

```

$T(n) = n + 1 + 1 + 1 + 1 + 1 + 1$
 $T(n) = n + 6$
 $T(n) = O(n)$

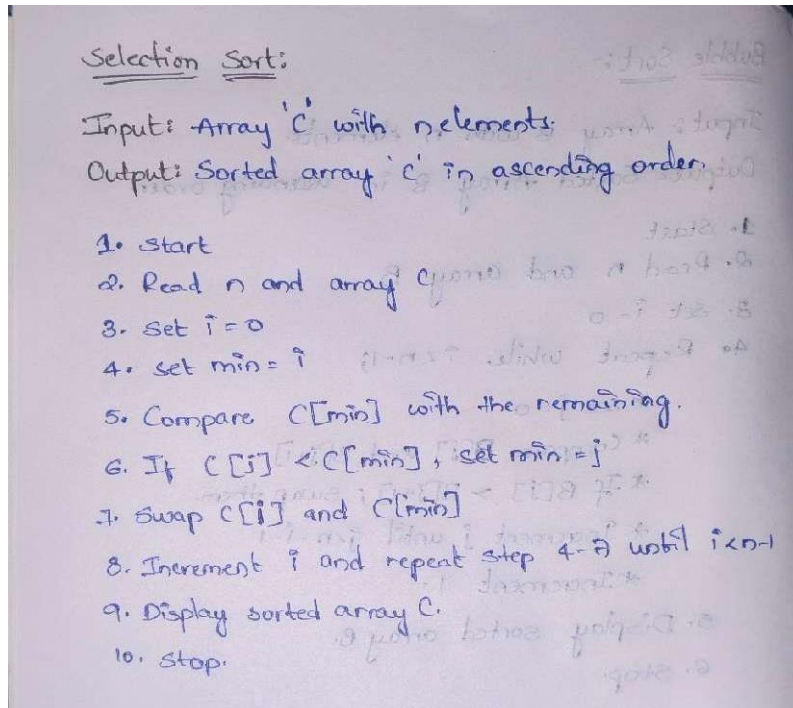
Best case: Perform all comparison only check every pair
 $(n-1) + (n-2) + \dots + 2 + 1$
 $T(n) = n(n-1)/2$
 $= (n^2 - n)/2$
 $T(n) = O(n^2)$

Average case:
 $T(n) = (n-1) + (n-2) + \dots + 1$
 $= n(n-1)/2$
 $T(n) = O(n^2)$

Worst case: all comparison
 $T(n) = O(n^2)$

B) Selection Sort:

Algorithm:

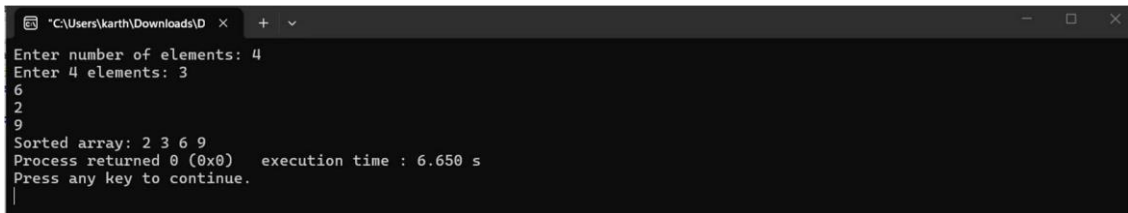


Program / Code:

```
#include <iostream>
using namespace std;
void selectionSort(int arr[], int n) {
    for (int i = 0; i < n - 1; i++) {
        int minIndex = i;
        for (int j = i + 1; j < n; j++) {
            if (arr[j] < arr[minIndex]) {
                minIndex = j;
            }
        }
        int temp = arr[minIndex];
```

```
arr[minIndex] = arr[i];  
arr[i] = temp;  
}  
}  
int main() {  
    int n;  
    cout << "Enter number of elements: ";  
    cin >> n;  
    int arr[n];  
    cout << "Enter " << n << " elements: ";  
    for (int i = 0; i < n; i++) {  
        cin >> arr[i];  
    }  
    selectionSort(arr, n);  
    cout << "Sorted array: ";  
    for (int i = 0; i < n; i++){  
        cout << arr[i] << " ";  
    }  
    return 0;  
}
```

Output:



```
"C:\Users\karth\Downloads\D" x + v  
Enter number of elements: 4  
Enter 4 elements: 3  
6  
2  
9  
Sorted array: 2 3 6 9  
Process returned 0 (0x0)   execution time : 6.650 s  
Press any key to continue.
```

Time complexity:

Selection Sort:

```

for (i=0; i<n-1; i++) {
    min = i;
    for (j=i+1; j<n; j++) {
        if (arr[j] < arr[min]) {
            min = j;
        }
    }
    temp = arr[min];
    arr[min] = arr[i];
    arr[i] = temp;
}
    
```

Time Complexity Analysis:

i	j	Number of comparisons
0	1 to n-1	n-1
1	2 to n-1	n-2
2	3 to n-1	n-3
...	...	...
n-2	n-1	1

Total comparisons = $(n-1) + (n-2) + (n-3) + \dots + 1$

$$T(n) = \frac{n(n-1)}{2} = \frac{n^2 - n}{2}$$

Time Complexity: $O(n^2)$

Best Case: array is sorted one. Comparison

$T(n) = n-1$

$T(n) = O(n)$

Worst Case:

$(n-1) + (n-2) + \dots + 1$

$T(n) = \frac{n(n-1)}{2}$

$T(n) = O(n^2)$

Average Case:

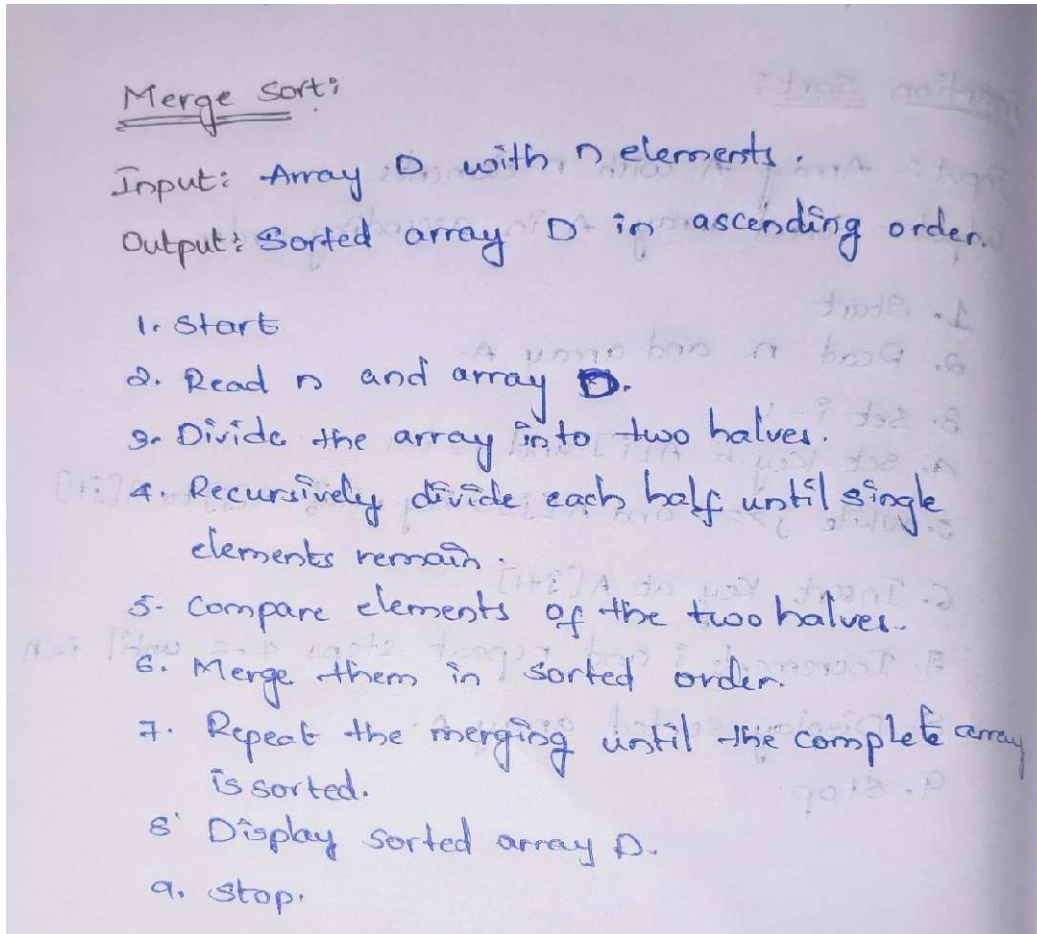
$1 + 2 + 3 + \dots + (n-1)$

$T(n) = \frac{n(n-1)}{2}$

$T(n) = O(n^2)$

C) Merge Sort:

Algorithm:



Program / Code:

```
#include <iostream>

using namespace std;

void merge(int arr[], int left, int mid, int right) {

    int n1 = mid - left + 1;

    int n2 = right - mid;

    int L[n1], R[n2];

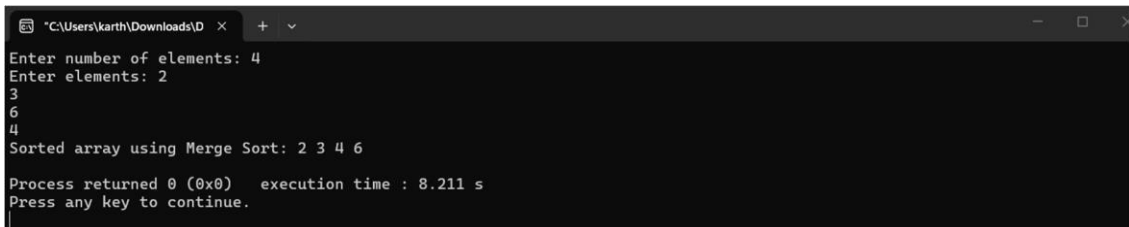
    for (int i = 0; i < n1; i++) {

        L[i] = arr[left + i];
```

```
    }  
    for (int j = 0; j < n2; j++) {  
        R[j] = arr[mid + 1 + j];  
    }  
    int i = 0;  
    int j = 0;  
    int k = left;  
    while (i < n1 && j < n2) {  
        if (L[i] <= R[j]) {  
            arr[k++] = L[i++];  
        } else {  
            arr[k++] = R[j++];  
        }  
    }  
    while (i < n1) {  
        arr[k++] = L[i++];  
    }  
    while (j < n2) {  
        arr[k++] = R[j++];  
    }  
}  
  
void mergeSort(int arr[], int left, int right) {  
    if (left < right) {  
        int mid = left + (right - left) / 2;  
        mergeSort(arr, left, mid);  
        mergeSort(arr, mid + 1, right);  
        merge(arr, left, mid, right);  
    }  
}
```

```
    }  
}  
  
int main() {  
    int n;  
    cout << "Enter number of elements: ";  
    cin >> n;  
    int arr[n];  
    cout << "Enter elements: ";  
    for (int i = 0; i < n; i++) {  
        cin >> arr[i];  
    }  
    mergeSort(arr, 0, n - 1);  
    cout << "Sorted array using Merge Sort: ";  
    for (int i = 0; i < n; i++) {  
        cout << arr[i] << " ";  
    }  
    cout << endl;  
    return 0;  
}
```

Output:



```
Enter number of elements: 4  
Enter elements: 2  
3  
6  
4  
Sorted array using Merge Sort: 2 3 4 6  
Process returned 0 (0x0) execution time : 8.211 s  
Press any key to continue.
```

Time Complexity:

Merge Sort

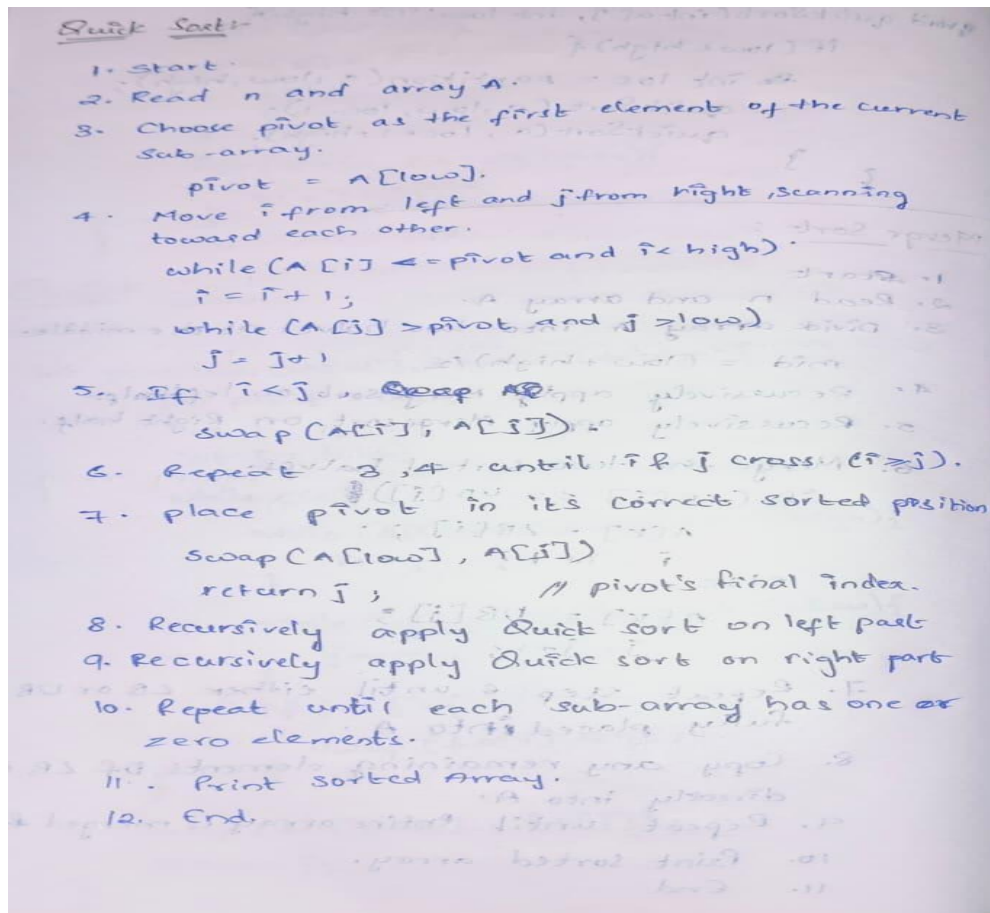
1. Start
2. Read n and array A .
3. Divide array A into two halves at the middle.
 $mid = (low + high) / 2$
4. Recursively apply MergeSort on left half.
5. Recursively apply MergeSort on right half.
6. Merge the two sorted halves.
if ($LB[i] \leq UB[j]$)
 $A[k] = LB[i]$;
 $i = i + 1$;
else
 $A[k] = UB[j]$;
 $j = j + 1$;
7. Repeat step 6 until either LB or UB is fully placed into A .
8. Copy any remaining elements of LB or UB directly into A .
9. Repeat until entire array is merged & sorted.
10. Print sorted array.
11. End.

(i) Best Case: Divided array into two halves
 $n \rightarrow n/2 + n/2 \rightarrow n/4 + n/4 \rightarrow \dots$
 $T(n) = n \times \log_2 n$
 $T(n) = O(n \log n)$

(ii) Average Case:
 $n \rightarrow n/2 \rightarrow n/4 \rightarrow \dots$
 $T(n) = n \times \log_2 n$
 $T(n) = O(n \log n)$

(iii) Worst Case: merge sort continuously divided the arrays into halves then merged all elements.
 $n \rightarrow n/2 \rightarrow n/4 \rightarrow \dots$
 $T(n) = n \times \log_2 n$
 $T(n) = O(n \log n)$

D) Quick Sort:



Algorithm:

Program / Code:

```
#include <iostream>
using namespace std;
int partition(int arr[], int lb, int ub) {
    int pivot = arr[lb];
    int start = lb;
    int end = ub;
    while (start < end) {
        while (arr[start] <= pivot) {
```

```
        start++;
    }
    while (arr[end] > pivot) {
        end--;
    }
    if (start < end) {
        swap(arr[start], arr[end]);
    }
}
swap(arr[lb], arr[end]);
return end;
}

void quickSort(int arr[], int lb, int ub) {
    if (lb < ub) {
        int loc = partition(arr, lb, ub);
        quickSort(arr, lb, loc - 1);
        quickSort(arr, loc + 1, ub);
    }
}

int main() {
    int n;

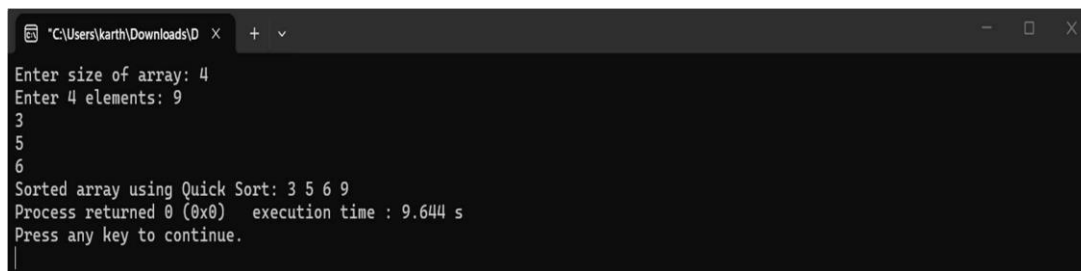
    cout << "Enter size of array: ";
    cin >> n;
    int arr[n];

    cout << "Enter " << n << " elements: ";
    for (int i = 0; i < n; i++) {
        cin >> arr[i];
    }

    quickSort(arr, 0, n - 1);
    cout << "Sorted array using Quick Sort: ";
    for (int i = 0; i < n; i++) {
        cout << arr[i] << " ";
    }
}
```

```
}  
return 0;  
}
```

Output:



```
*C:\Users\karth\Downloads\ID x + v  
Enter size of array: 4  
Enter 4 elements: 9  
3  
5  
6  
Sorted array using Quick Sort: 3 5 6 9  
Process returned 0 (0x0) execution time : 9.644 s  
Press any key to continue.  
|
```

Time Complexity:

Quick Sort:

1. Start.
2. Read n and array A .
3. Choose pivot as the first element of the current sub-array.
 $\text{pivot} = A[\text{low}]$.
4. Move i from left and j from right scanning toward each other.
while $(A[i] \leq \text{pivot and } i < \text{high})$.
 $i = i + 1$;
while $(A[j] > \text{pivot and } j \geq \text{low})$.
 $j = j - 1$;
5. If $i < j$ swap $A[i]$ and $A[j]$.
swap $(A[i], A[j])$.
6. Repeat 3, 4 until i & j cross ($i \geq j$).
7. place pivot in its correct sorted position.
swap $(A[\text{low}], A[i])$.
8. return i // pivot's final index.
9. Recursively apply Quick sort on left part.
10. Recursively apply Quick sort on right part.
11. Repeat until each sub-array has one or zero elements.
12. Print sorted Array.
13. End.

5. (i) Best Case: Pivot divides the array into two equal parts
 $n \rightarrow n/2 + n/2 \rightarrow n/4 + n/4 + n/4 + n/4 \rightarrow \dots$
 $T(n) = n \times \log_2 n$
 $T(n) = O(n \log n)$

(ii) Average Case: Divides into balance parts.
 $n \rightarrow n/2 + n/2 \rightarrow n/4 + n/4 \dots$
 $T(n) = n \times \log_2 n$
 $T(n) = O(n \log n)$

(iii) Worst case: Pivot is always smallest / largest elements.
 $n \rightarrow (n-1) \rightarrow (n-2) + (n-3) \rightarrow \dots + 1$
 $T(n) = (n-1) + (n-2) + \dots + 1 + 1$
 $T(n) = n(n-1)/2$
 $= (n^2 - n)/2$